ZIG

Первая презентация

История и особенности · Запуск программы · Данные и память · Функции · Поток управления · Ввод-вывод



Команда 16
Медведьков Роман
Крутянский Роман
Гуцалов Егор

Группа 5030102/30201

История Zig

Проект развивается с 2015 года и остаётся до версии 1.0 в активной эволюции.

2015

Начало проекта

Создатель — Andrew Kelley. Цель проекта — простой системный язык и инструментальная цепочка без скрытого поведения.

2020–2025

Язык, компилятор и build system активно стабилизируются

2026

Zig 0.17.0

Текущий стабильный релиз на момент подготовки презентации — 0.17.0.

Особенности и спецификация

Zig делает важные действия явными и старается минимизировать скрытое поведение.

01

Нет скрытых аллокаций

Динамическая память выделяется через явно переданный allocator.

02

Нет скрытого control flow

Управление выполнением выражено ключевыми словами и вызовами функций.

03

comptime

Часть вычислений и метапрограммирование выполняются во время компиляции.

04

C interoperability

Компилятор и toolchain ориентированы на интеграцию с C и кросс-компиляцию.

Последний стабильный релиз

`0.17.0` Релиз 2 октября 2026 года · язык всё ещё развивается до 1.0

Способы запуска программы

Для учебных примеров достаточно трёх сценариев: контейнер, браузер или локальная установка.

Docker

Плюсы

- Одинаковое окружение
- Изоляция от системы

Минус

Нужен Docker

Онлайн-среда

Плюсы

- Ничего не нужно ставить
- Быстрый старт

Минус

Есть ограничения

Локально

Плюсы

- Полный контроль
- Удобно для проектов

Минус

Нужна установка

Запуск через Docker

Контейнер удобен, когда нужно гарантировать одинаковую версию Zig у всех участников.

- Фиксируем версию компилятора в образе.
- Монтируем папку проекта внутрь контейнера.
- Запускаем zig run или zig build-exe внутри контейнера.

Dockerfile

```
FROM debian:bookworm-slim

ARG ZIG_VERSION=0.17.0

RUN apt-get update && apt-get install -y curl xz-utils \
    && curl -L \
    https://ziglang.org/download/${ZIG_VERSION}/zig-x86_64-linux-${ZIG_VERSION}.tar.xz \
    -o /tmp/zig.tar.xz \
    && tar -xf /tmp/zig.tar.xz -C /opt \
    && ln -s /opt/zig-x86_64-linux-${ZIG_VERSION}/zig /usr/local/bin/zig

WORKDIR /app
COPY . .

CMD ["zig", "run", "05_main.zig"]
```

05_main.zig

```
const std = @import("std");

pub fn main() void {
    std.debug.print("Hello, Zig!\n", .{});
}
```

```
docker run …  zig run main.zig
```

Онлайн-среда и локальное окружение

Для быстрых примеров подойдёт браузер; для полноценной работы удобнее локальная установка.

A

Онлайн

Playground: вставить код → Run.



B

Локально

Скачать Zig для своей платформы, добавить zig в PATH, проверить версию и запускать файлы из терминала.

```
zig version                zig run main.zig                zig build-exe main.zig
```

Типы данных

Zig использует статическую типизацию и явно различает значения, массивы, slices и optional-типы.

Integer

u8 · i32 · usize

Float

f32 · f64

Boolean

bool

Array

[N]T

Slice

[]T / []const T

Optional

?T

07_types.zig

```zig
const std = @import("std");

pub fn main() void {
    const age: u8 = 20;
    const temperature: f64 = 23.5;
    const active: bool = true;

    const numbers = [_]i32{ 10, 20, 30 };
    const slice: []const i32 = numbers[0..2];

    const nickname: ?[]const u8 = "Zig";

    std.debug.print("age = {d}\n", .{age});
    std.debug.print("temperature = {d:.1}\n", .{temperature});
    std.debug.print("active = {}\n", .{active});
    std.debug.print("array = {any}\n", .{numbers});
    std.debug.print("slice = {any}\n", .{slice});
    std.debug.print("optional = {s}\n", .{nickname.?});
}
```

Операции над типами

Базовые операции делятся на арифметические, логические и операции сравнения.

01

Арифметика

+ – * / %

02

Сравнение

== != < > <= >=

03

Логика

and or !

```zig
const std = @import("std");

pub fn main() void {
    const a: i32 = 10;
    const b: i32 = 3;

    std.debug.print("a + b = {d}\n", .{a + b});
    std.debug.print("a - b = {d}\n", .{a - b});
    std.debug.print("a * b = {d}\n", .{a * b});
    std.debug.print("a > b = {}\n", .{a > b});
    std.debug.print("a == b = {}\n", .{a == b});

    const ready = true;
    const allowed = false;
    std.debug.print("ready and allowed = {}\n", .{ready and allowed});
}
```

ВЫВОД

```
13
true
```

Преобразование типов

Потенциально опасные преобразования в Zig обычно делаются явно.

@intCast

Преобразование между целочисленными типами.

@floatFromInt

Преобразование integer → float.

@intFromFloat

Преобразование float → integer.

09_casts.zig

```zig
const std = @import("std");

pub fn main() void {
    const small: u8 = 42;
    const wide: u32 = small;

    const signed_value: i32 = 100;
    const narrow: u8 = @intCast(signed_value);

    const integer: i32 = 25;
    const real: f64 = @floatFromInt(integer);

    const pi: f64 = 3.99;
    const truncated: i32 = @intFromFloat(pi);

    std.debug.print("wide = {d}\n", .{wide});
    std.debug.print("narrow = {d}\n", .{narrow});
    std.debug.print("real = {d:.1}\n", .{real});
    std.debug.print("truncated = {d}\n", .{truncated});
}
```

Области видимости

Объявления могут быть глобальными, локальными или существовать только внутри блока.

- const — значение нельзя переназначить.
- var — значение может изменяться.
- Вложенный блок создаёт собственную область видимости.
- Имена объявлений должны быть однозначны внутри своей области.

```
const std = @import("std");

const global_value: i32 = 100;

pub fn main() void {
    const x: i32 = 10;
    var counter: i32 = 0;

    {
        const y: i32 = 20;
        counter += 1;
        std.debug.print("inside: x={d}, y={d}\n", .{ x, y });
    }

    std.debug.print("global={d}\n", .{global_value});
    std.debug.print("x={d}, counter={d}\n", .{ x, counter });
}
```

Владение и управление памятью

Zig не скрывает выделение памяти: allocator передаётся явно, а освобождение ресурса видно в коде.

01 **Значение**

Небольшие значения могут передаваться и копироваться.

02 **Указатель**

Работа с существующим объектом без копирования содержимого.

03 **Allocator + defer**

Выделение и освобождение памяти остаются явными в коде.

```zig
const std = @import("std");

pub fn main() !void {
    const allocator = std.heap.page_allocator;

    const memory = try allocator.alloc(u8, 10);
    defer allocator.free(memory);

    @memset(memory, 0);

    memory[0] = 42;
    memory[1] = 7;

    std.debug.print("len = {d}\n", .{memory.len});
    std.debug.print("values = {d}, {d}\n", .{ memory[0], memory[1] });
}
```

Функции: входные и выходные данные

Типы параметров и возвращаемого значения являются частью объявления функции.

IN

Параметры

Каждый параметр объявляется с именем и типом.

OUT

Результат

Тип после списка параметров задаёт возвращаемое значение; void означает отсутствие полезного результата.

```zig
const std = @import("std");

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn greet(name: []const u8) void {
    std.debug.print("Hello, {s}!\n", .{name});
}

pub fn main() void {
    const result = add(5, 7);

    std.debug.print("result = {d}\n", .{result});
    greet("Zig");
}
```

ВЫВОД

```
12
```

Рекурсия

Функция может вызывать саму себя, пока не выполнится условие остановки.

factorial(5)

```
5 × factorial(4)
↓
5 × 4 × factorial(3)
↓
5 × 4 × 3 × 2 × 1
↓
120
```

```zig
const std = @import("std");

fn factorial(n: u32) u32 {
    if (n <= 1) {
        return 1;
    }

    return n * factorial(n - 1);
}

pub fn main() void {
    const result = factorial(5);
    std.debug.print("factorial(5) = {d}\n", .{result});
}
```

ВЫВОД

```
120
```

Замыкания и передача контекста

Связанное с функцией состояние можно хранить явно и передавать через структуру и указатель на неё.

- Состояние хранится в полях структуры.
- Метод получает *Self и изменяет текущий объект.
- Такой приём удобно использовать для счётчиков, callback-контекста и итераторов.

```zig
const std = @import("std");

const Counter = struct {
    value: i32,

    fn next(self: *Counter) i32 {
        self.value += 1;
        return self.value;
    }
};

pub fn main() void {
    var counter = Counter{ .value = 0 };

    std.debug.print("{d}\n", .{counter.next()});
    std.debug.print("{d}\n", .{counter.next()});
    std.debug.print("{d}\n", .{counter.next()});
}
```

ВЫВОД

```
1 · 2 · 3
```

Условные конструкции

Основные конструкции — if / else и switch. Они могут участвовать в вычислении значения.

IF

if / else

Подходит для проверки одного или нескольких логических условий.

SW

switch

Удобен, когда значение нужно сопоставить с несколькими вариантами.

```zig
const std = @import("std");

pub fn main() void {
    const score: u8 = 84;

    if (score >= 60) {
        std.debug.print("Passed\n", .{});
    } else {
        std.debug.print("Failed\n", .{});
    }

    const level: u8 = 2;

    switch (level) {
        1 => std.debug.print("beginner\n", .{}),
        2 => std.debug.print("intermediate\n", .{}),
        3 => std.debug.print("advanced\n", .{}),
        else => std.debug.print("unknown\n", .{}),
    }
}
```

Циклы

Для повторения действий используются while и for; for удобно обходит массивы и slices.

01

while

Повторяет тело, пока условие истинно. После тела можно указать выражение обновления.

02

for

Обходит элементы последовательности и может одновременно выдавать индекс.

```zig
const std = @import("std");

pub fn main() void {
    var i: u8 = 0;

    while (i < 3) : (i += 1) {
        std.debug.print("while: {d}\n", .{i});
    }

    const numbers = [_]i32{ 10, 20, 30 };

    for (numbers, 0..) |number, index| {
        std.debug.print("for: [{d}] = {d}\n", .{ index, number });
    }
}
```

Итераторы

Один из типичных паттернов — объект хранит своё состояние и возвращает следующий элемент через next().

- current — текущая позиция.
- end — граница последовательности.
- next() возвращает ?T: значение либо null, когда элементы закончились.
- while может извлекать значение из optional прямо в условии.

```zig
const std = @import("std");

const Counter = struct {
    current: usize,
    end: usize,

    fn next(self: *Counter) ?usize {
        if (self.current >= self.end) {
            return null;
        }

        const value = self.current;
        self.current += 1;
        return value;
    }
};

pub fn main() void {
    var counter = Counter{
        .current = 0,
        .end = 4,
    };

    while (counter.next()) |value| {
        std.debug.print("{d}\n", .{value});
    }
}
```

Блоки

Именованный блок может вернуть значение через break :label value.

```
blk: { … }

↓

break :blk value

↓

result
```

```zig
const std = @import("std");

pub fn main() void {
    const result = blk: {
        const a = 10;
        const b = 20;

        break :blk a + b;
    };

    std.debug.print("result = {d}\n", .{result});
}
```

ВЫВОД

```
30
```

Работа с консолью

Стандартная библиотека предоставляет средства для вывода и чтения данных через стандартные потоки.

OUT

Вывод

Для учебных примеров удобно использовать std.debug.print; форматная строка проверяется компилятором.

IN

Ввод

Чтение выполняется через API стандартного ввода и reader; конкретный пример проверим на Zig 0.17.0.

```zig
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const io = init.io;

    var stdin_buffer: [256]u8 = undefined;
    var stdin_reader = std.Io.File.stdin().reader(io, &stdin_buffer);
    const stdin = &stdin_reader.interface;

    var stdout_buffer: [256]u8 = undefined;
    var stdout_writer = std.Io.File.stdout().writer(io, &stdout_buffer);
    const stdout = &stdout_writer.interface;

    try stdout.print("Enter your name: ", .{});
    try stdout.flush();

    const name = try stdin.takeDelimiter('\n') orelse return;

    try stdout.print("Hello, {s}!\n", .{name});
    try stdout.flush();
}
```

Работа с файлами

Файл открывается или создаётся явно, операции могут вернуть ошибку, а ресурс нужно закрыть.

- std.fs — работа с файловой системой.
- try — передаёт ошибку вызывающему коду.
- defer file.close() — гарантирует закрытие файла при выходе из текущей области.
- Чтение и запись выполняются через методы файла / reader / writer.

```zig
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    const io = init.io;
    const cwd = std.Io.Dir.cwd();

    {
        const file = try cwd.createFile(io, "example.txt", .{
            .truncate = true,
        });
        defer file.close(io);

        try file.writeStreamingAll(io, "Hello, Zig!");
    }

    {
        const file = try cwd.openFile(io, "example.txt", .{});
        defer file.close(io);

        var buffer: [64]u8 = undefined;
        const bytes_read = try file.readPositionalAll(io, &buffer, 0);

        std.debug.print(
            "File content: {s}\n",
            .{buffer[0..bytes_read]},
        );
    }
}
```

Что разобрали

Первая презентация даёт базовую картину языка и показывает, как запустить и написать небольшую программу на Zig.

01

Обзор

история · особенности · 0.17.0

02

Запуск

Docker · online · local

03

Данные

типы · cast · scope · память

04

Функции

параметры · рекурсия · контекст

05

Control flow

if · switch · while · for · блоки

06

I/O

консоль · файлы

Спасибо за внимание